

Informatika 3

Prednáška 3: Práca s pamäťou

Michal Mrena



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Typ ukazovateľ

Angl. pointer



Typ ukazovateľ

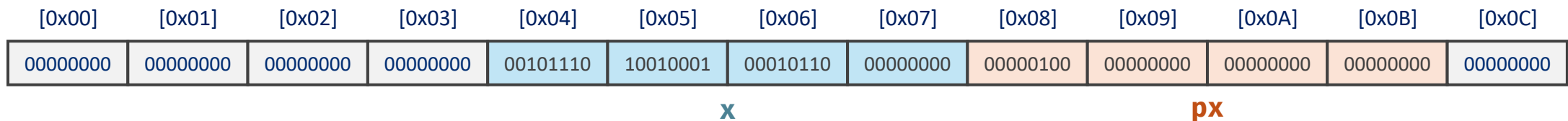
- Premenná typu ukazovateľ (angl. pointer) je **premenná ako každá iná**
- Jej hodnota predstavuje **adresu** v pamäti, na ktorej sa nachádza hodnota *ukazovaného typu* (angl. *pointee/pointed to type*)
- Syntax: <typ> *
- Príklady:
 - `int *` – ukazovateľ na typ `int`
 - `double *` – ukazovateľ na typ `double`
 - `int **` – ukazovateľ na typ ukazovateľ na typ `int`

Typ ukazovateľ

- Adresa je prakticky index bajtu v pamäti
 - Často ju vidíme zapísanú v šestnástkovej sústave – čo na fakte, že ide o obyčajné číslo, nič nemení
- V závislosti od dátového modelu má 4 alebo 8 bajtov

V príklade uvažujeme little-endian architektúru a veľkosť ukazovateľa 4B.

```
int main() {  
    int x = 1'478'958;  
    int *px = &x;  
}
```



Základné operácie

Unárny operátor &

- Získa adresu argumentu
- Napríklad: Ak je argumentom premenná typu `int`, vráti hodnotu typu `int *`

Unárny operátor *

- Sprístupní *hodnotu*, ktorej adresu v sebe argument (ukazovateľ) uchováva
- Je možné priradiť hodnotu na adresované miesto
- Operáciu nazývame aj **dereferencovanie**
- Je potrebné rozlišovať, či je znak `*` použitý ako súčasť typu alebo súčasť výrazu

```
#include <iostream>

void add(int *pa, int *pb, int *result) {
    int a = *pa;
    int b = *pb;
    *result = a + b;
}

int main() {
    int a = 20;
    int b = 30;
    int c;
    add(&a, &b, &c);
    std::cout << c << "\n";
    // 50
}
```

Typ ukazovateľ

- Ukazovatele na rôzne typy sú rôzne typy
- Niektoré typy ukazovateľov sú medzi sebou implicitne konvertovateľné

```
int main() {  
    int x = 1'478'958;  
    double *px = &x;  
}
```

```
/main.cpp:5:18: error:  
cannot convert 'int*' to 'double*' in initialization
```

Nulový ukazovateľ

- Naprieč rôznymi programovacími jazykmi poznáme koncept tzv. **nulového ukazovateľa** (angl. **null pointer**) –, ktorý neukazuje na žiadne pamäťové miesto*
- Napr. v jazyku Java môžu referenčné premenné nadobúdať hodnotu `null`
- V jazyku C++ túto hodnotu vyjadrujeme literálom typu ukazovateľ `nullptr`

```
void printInt(int *px) {  
    if (px != nullptr) {  
        std::cout << *px << "\n";  
    }  
}  
  
int main() {  
    int *px = nullptr;  
    printInt(px);  
}
```

* Tento koncept jeho autor označil ako chybu za miliardy dolárov

Nulový ukazovateľ

- Na internete a v praxi sa stretnete s použitím macra `NULL` namiesto literálu `nullptr`
- Použitie macra, aj keď sa vo *väčšine* prípadov bude správať rovnako/podobne, má niekoľko nevýhod, ktoré pramenia z toho, že macro sa iba expanduje na **celočíselnú konštantu** (zvyčajne `0` alebo `0L`)

```
void printInt(int *px) {  
    if (px != NULL) {  
        std::cout << *px << "\n";  
    }  
}  
  
int main() {  
    int *px = NULL;  
    printInt(px);  
}
```

- Preto, vždy používame literál `nullptr`

Ukazovateľ a implicitné konverzie

- Ukazovateľ je **implicitne konvertovateľný** na typ `bool`
- Preto na miestach, kde sa očakáva hodnota typu `bool` (`if`, `while`, ...), môžeme použiť aj hodnotu typu ukazovateľ
- S takýmto zápisom sa bežne stretneme

```
void printInt(int *px) {  
    if (px) {  
        std::cout << *px << "\n";  
    }  
}  
  
int main() {  
    int *px = nullptr;  
    printInt(px);  
}
```

- Konverzia je nasledovná:

`nullptr` -> `false`
inak -> `true`

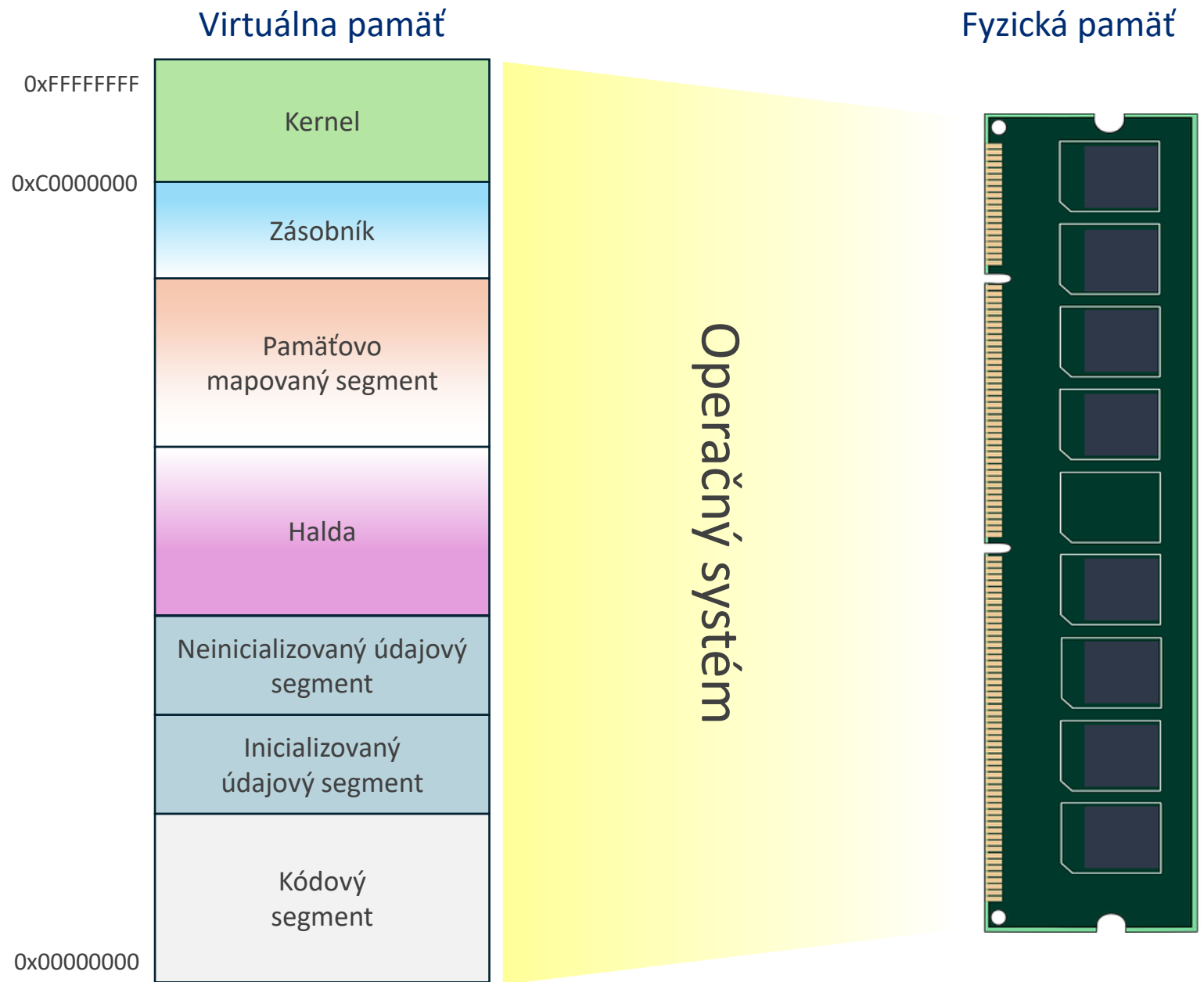
Pamäťový priestor



Pamäťový priestor

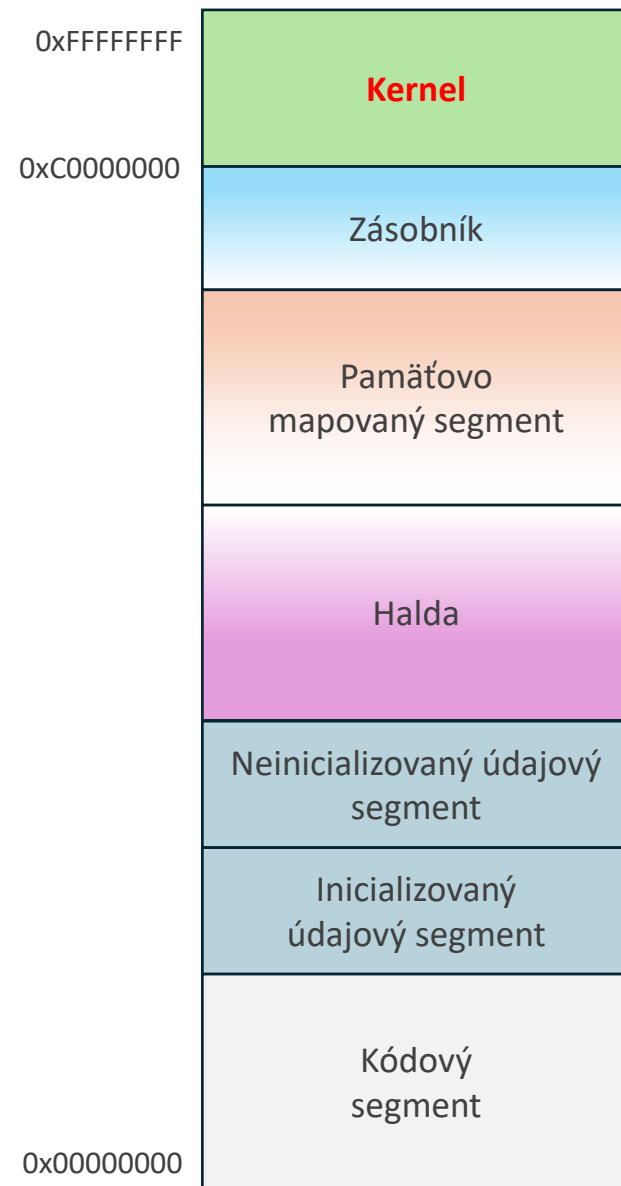
- Po skompilovaní sa **program** (pasívna entita) nachádza na **disku**
- Jeho spustenie – zavedenie do **operačnej pamäte** a ďalšiu réžiu – zabezpečuje **operačný systém**
- Spustením sa z neho stáva **proces** (aktívna entita)
- Pohľad na pamäť ako **veľké pole bajtov** je stále korektný
- Pamäť je z pohľadu procesu rozdelená na **segmenty**
- Rôzne segmenty uchovávajú údaje rôznej povahy

- Proces pracuje s virtuálnymi adresami
- Preklad na fyzické adresy zabezpečuje operačný systém
- Preklad je pre programátora transparentný



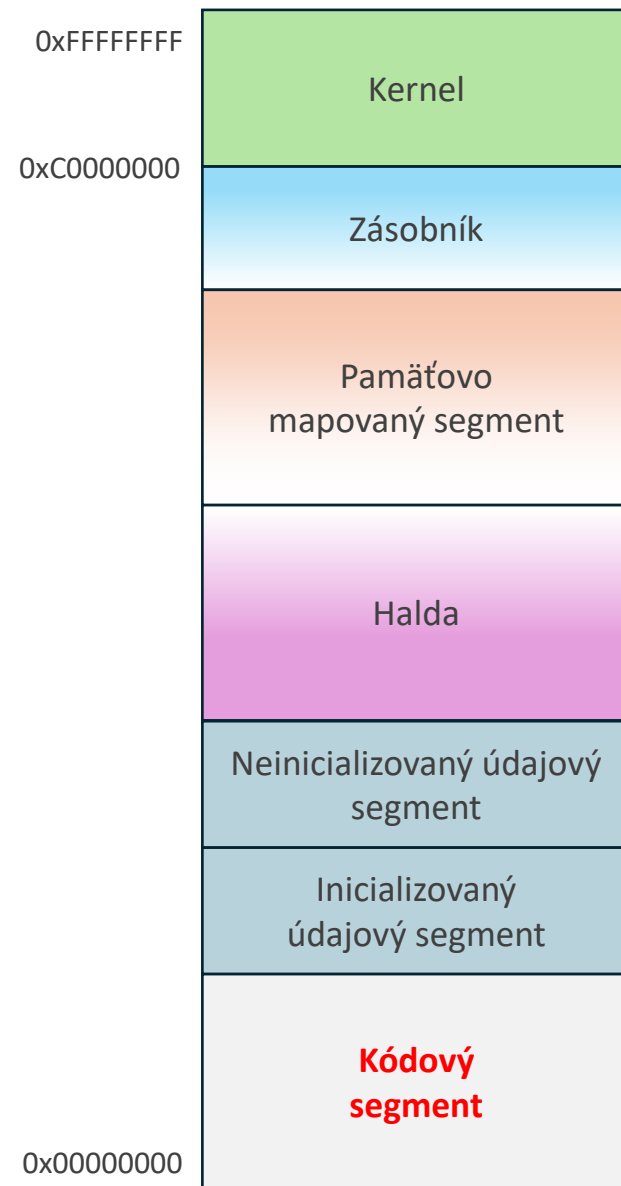
Kernel

- Časť vyhradená pre operačný systém
- Proces k nej nemá priamy prístup



Kódový segment

- Obsahuje inštrukcie metód a funkcií
- Môžu pri ňom byť uložené literály
- Register RIP (x86-64) (vo všeobecnosti tzv. program counter) obsahuje adresu inštrukcie v tomto segmente
- Pre proces je tento segment **iba na čítanie**



```

#include <iostream>

int square(int n) {
    return n * n;
}

int main() {
    std::cout << "Enter a number: ";
    int n;
    std::cin >> n;
    const int s = square(n);
    std::cout << "Square of your number is: "
              << n << "\n";
}

```

```

square(int):
    imul    edi, edi
    mov     eax, edi
    ret

.LC0:
    .string "Enter a number: "
.LC1:
    .string "Square of your number is: "
.LC2:
    .string "\n"
main:
    sub     rsp, 24
    mov     edx, 16
    mov     esi, OFFSET FLAT:.LC0
    mov     edi, OFFSET FLAT:std::cout
    call    std::basic_ostream ...
    lea     rsi, [rsp+12]
    mov     edi, OFFSET FLAT:std::cin
    call    std::istream::operator>>(int&)
    mov     edx, 26
    mov     esi, OFFSET FLAT:.LC1
    mov     edi, OFFSET FLAT:std::cout
    call    std::basic_ostream ...
    mov     esi, DWORD PTR [rsp+12]
    mov     edi, OFFSET FLAT:std::cout
    call    std::ostream::operator<<(int)
    mov     edx, 1
    mov     esi, OFFSET FLAT:.LC2
    mov     rdi, rax
    call    std::basic_ostream ...
    xor     eax, eax
    add     rsp, 24
    ret

```

Údajový segment

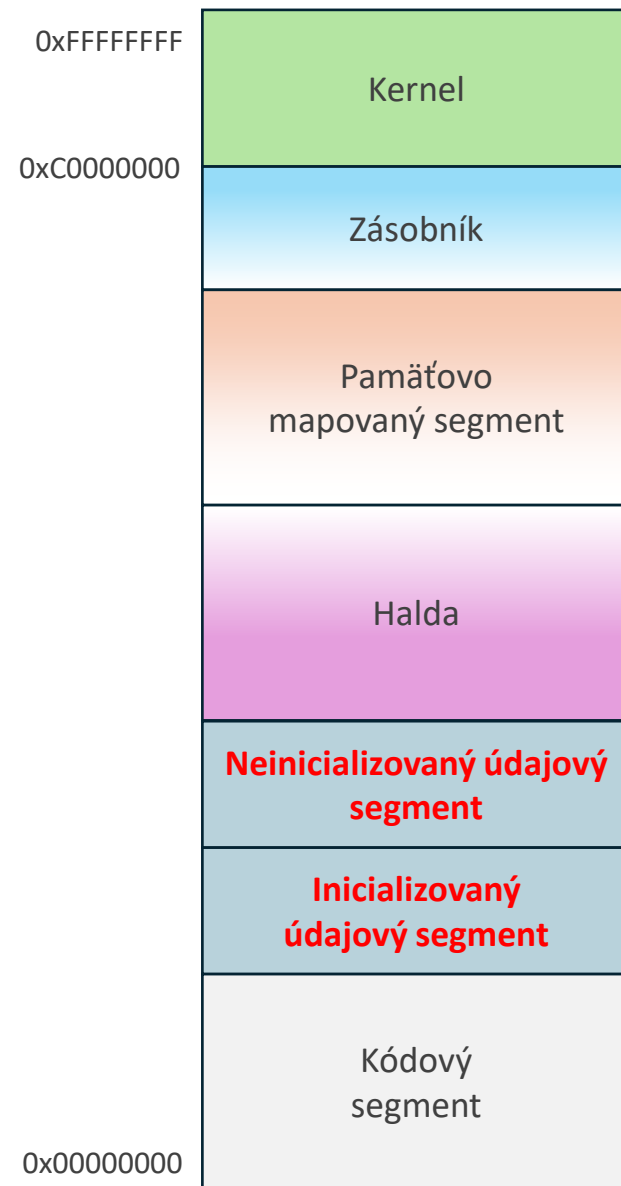
- Obsahuje globálne a statické premenné

```
#include <iostream>

int n; // Neinicializovaný údajový segment

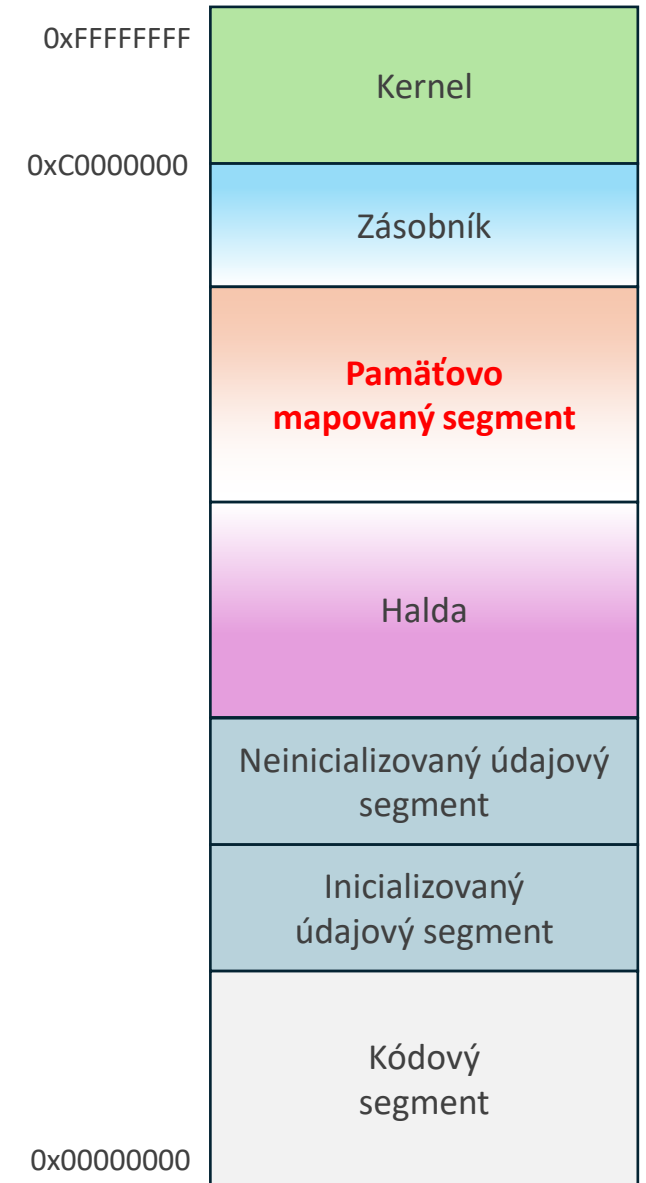
void printNumber() {
    static int i = 0; // Inicializovaný údajový segment
    std::cout << i++ << "\n";
}

int main() {
    std::cin >> n;
    for (int i = 0; i < n; ++i) {
        printNumber();
    }
}
```



Pamäťovo mapovaný segment

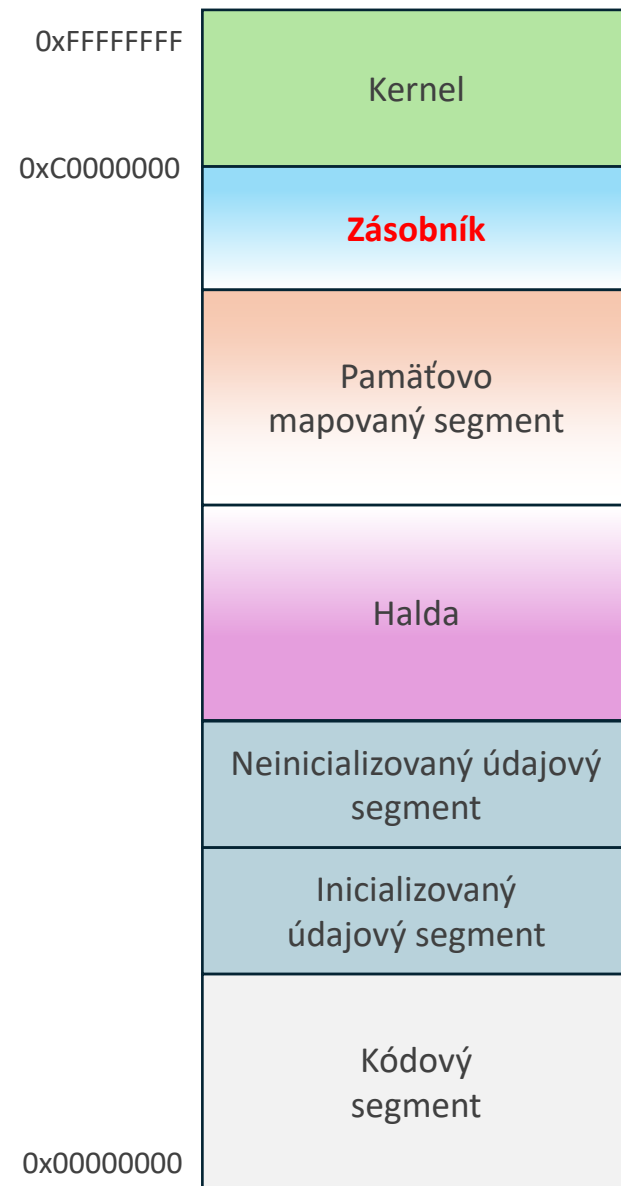
- Slúži na pokročilú prácu s pamäťou
- Bližšie sa mu budeme venovať na predmete *6BI0035 princípy operačných systémov*
- Môže obsahovať
 - Dynamicky linkované knižnice
 - Zdieľanú pamäť
 - Pamäťovo mapované súbory



Zásobník

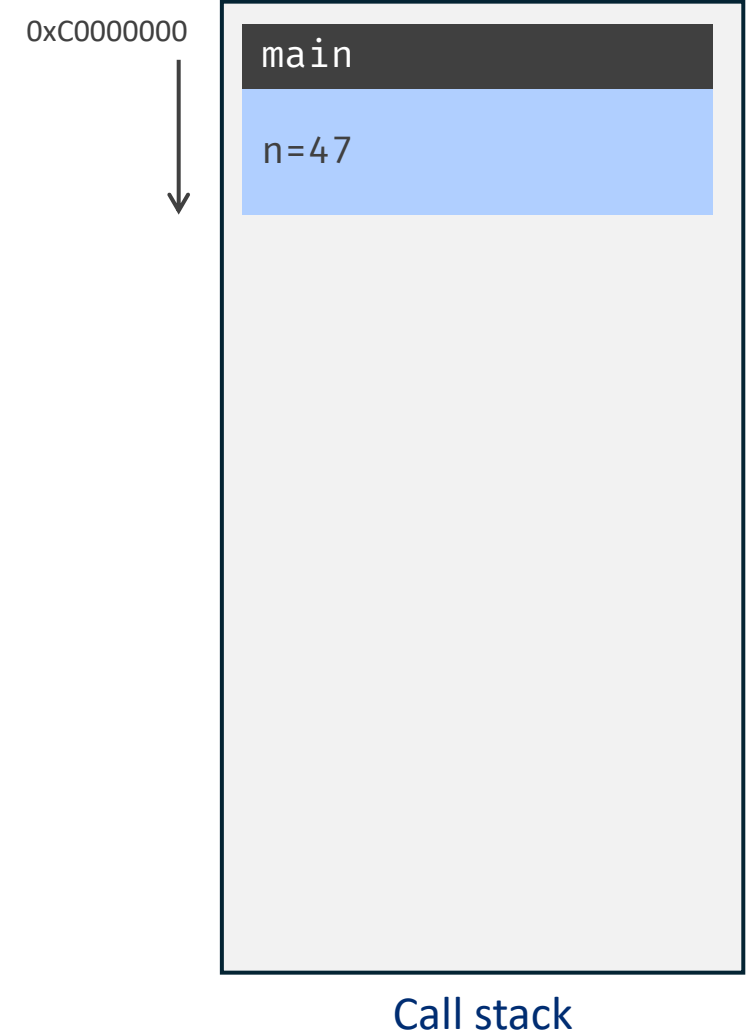
- Obsahuje **rámce**
- Jeden rámec zodpovedá jednému volaniu funkcie/metódy a obsahuje:
 - Lokálne premenné (definované vo funkcii)
 - Parametre *
 - Návratovú adresu (adresa inštrukcie, ktorá nasleduje po ukončení funkcie)
 - Návratovú hodnotu *
- Veľkosť rámca je známa už pri preklade

* Odovzdávanie týchto hodnôt môže byť kompilátorom optimalizované použitím registrov



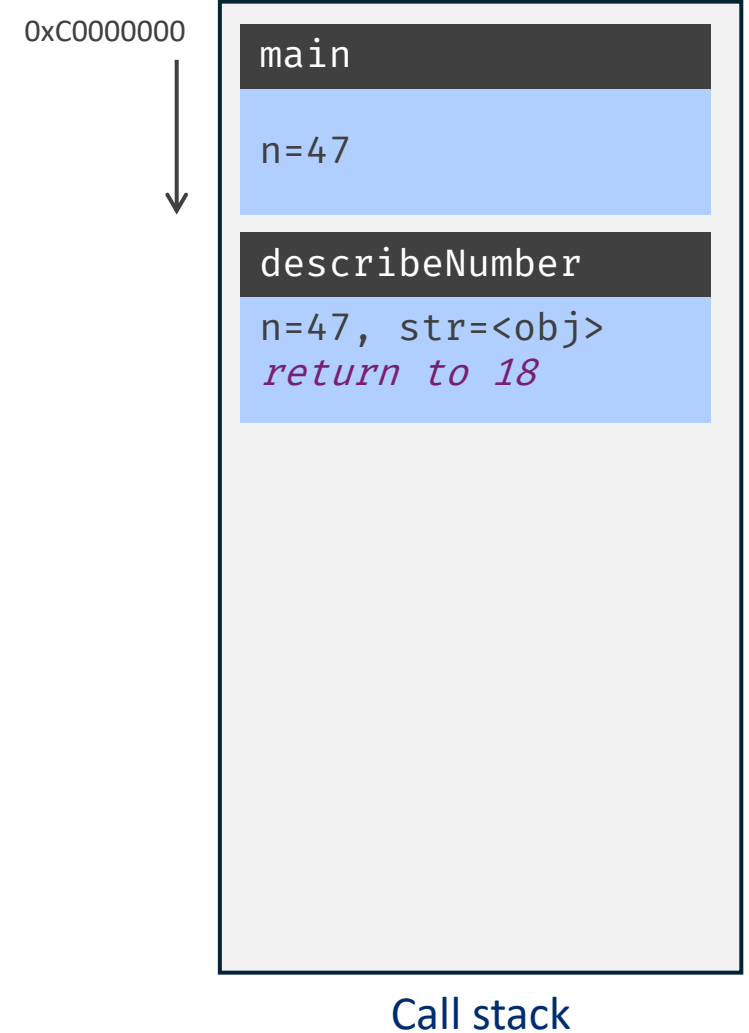
Zásobník – call stack

```
1 #include <iostream>
2
3 bool isOdd(int n) {
4     bool is = n == 1;
5     return is;
6 }
7 void describeNumber(int n) {
8     std::string str;
9     if (isOdd(n & 1)) {
10         str = " is odd\n";
11     } else {
12         str = " is even\n";
13     }
14     std::cout << n << str;
15 }
16 int main() {
17     int n = 47;
18     describeNumber(n);
19 }
```



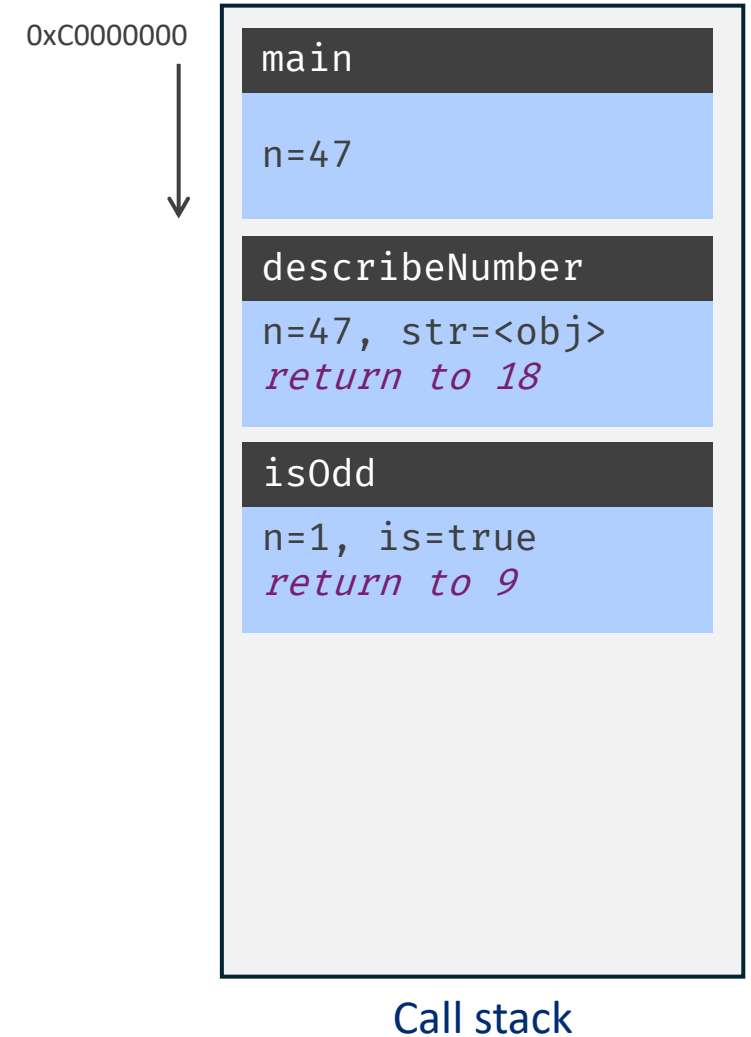
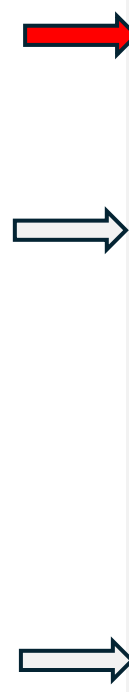
Zásobník – call stack

```
1 #include <iostream>
2
3 bool isOdd(int n) {
4     bool is = n == 1;
5     return is;
6 }
7 void describeNumber(int n) {
8     std::string str;
9     if (isOdd(n & 1)) {
10         str = " is odd\n";
11     } else {
12         str = " is even\n";
13     }
14     std::cout << n << str;
15 }
16 int main() {
17     int n = 47;
18     describeNumber(n);
19 }
```



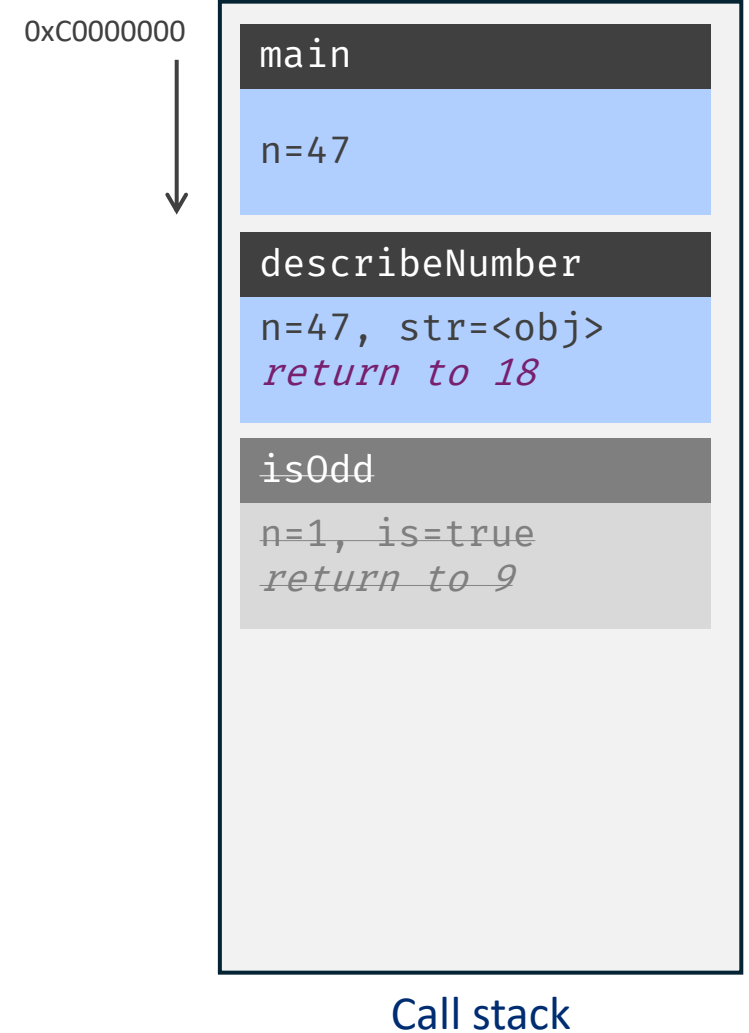
Zásobník – call stack

```
1 #include <iostream>
2
3 bool isOdd(int n) {
4     bool is = n == 1;
5     return is;
6 }
7 void describeNumber(int n) {
8     std::string str;
9     if (isOdd(n & 1)) {
10         str = " is odd\n";
11     } else {
12         str = " is even\n";
13     }
14     std::cout << n << str;
15 }
16 int main() {
17     int n = 47;
18     describeNumber(n);
19 }
```



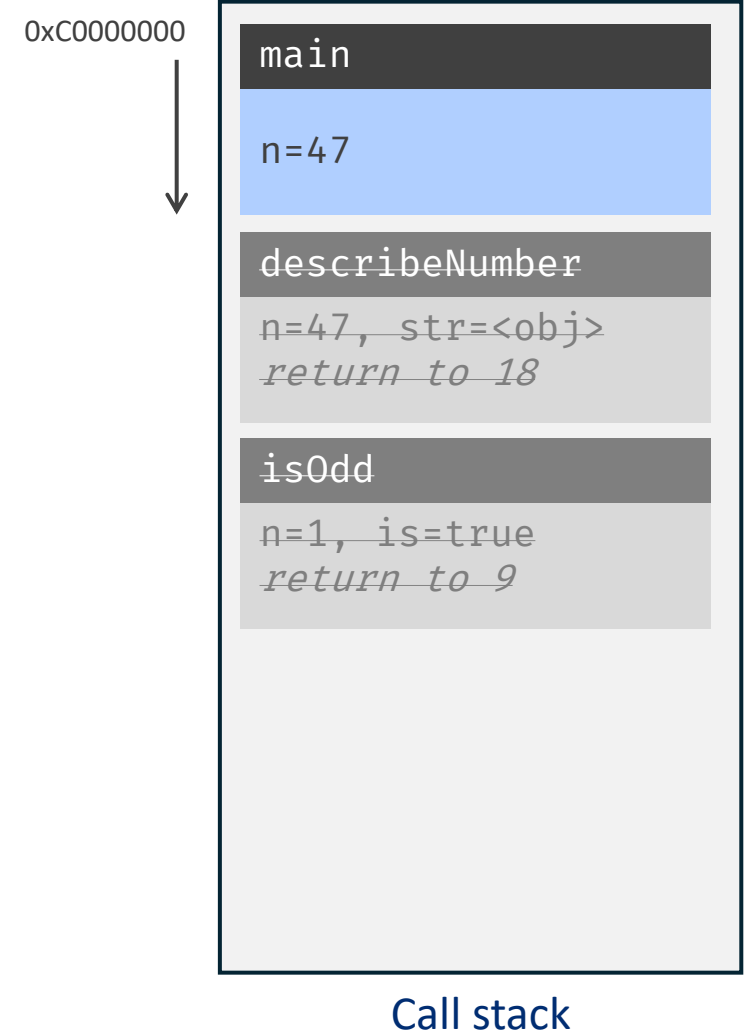
Zásobník – call stack

```
1 #include <iostream>
2
3 bool isOdd(int n) {
4     bool is = n == 1;
5     return is;
6 }
7 void describeNumber(int n) {
8     std::string str;
9     if (isOdd(n & 1)) {
10        str = " is odd\n";
11    } else {
12        str = " is even\n";
13    }
14    std::cout << n << str;
15 }
16 int main() {
17     int n = 47;
18     describeNumber(n);
19 }
```



Zásobník – call stack

```
1 #include <iostream>
2
3 bool isOdd(int n) {
4     bool is = n == 1;
5     return is;
6 }
7 void describeNumber(int n) {
8     std::string str;
9     if (isOdd(n & 1)) {
10         str = " is odd\n";
11     } else {
12         str = " is even\n";
13     }
14     std::cout << n << str;
15 }
16 int main() {
17     int n = 47;
18     describeNumber(n);
19 }
```

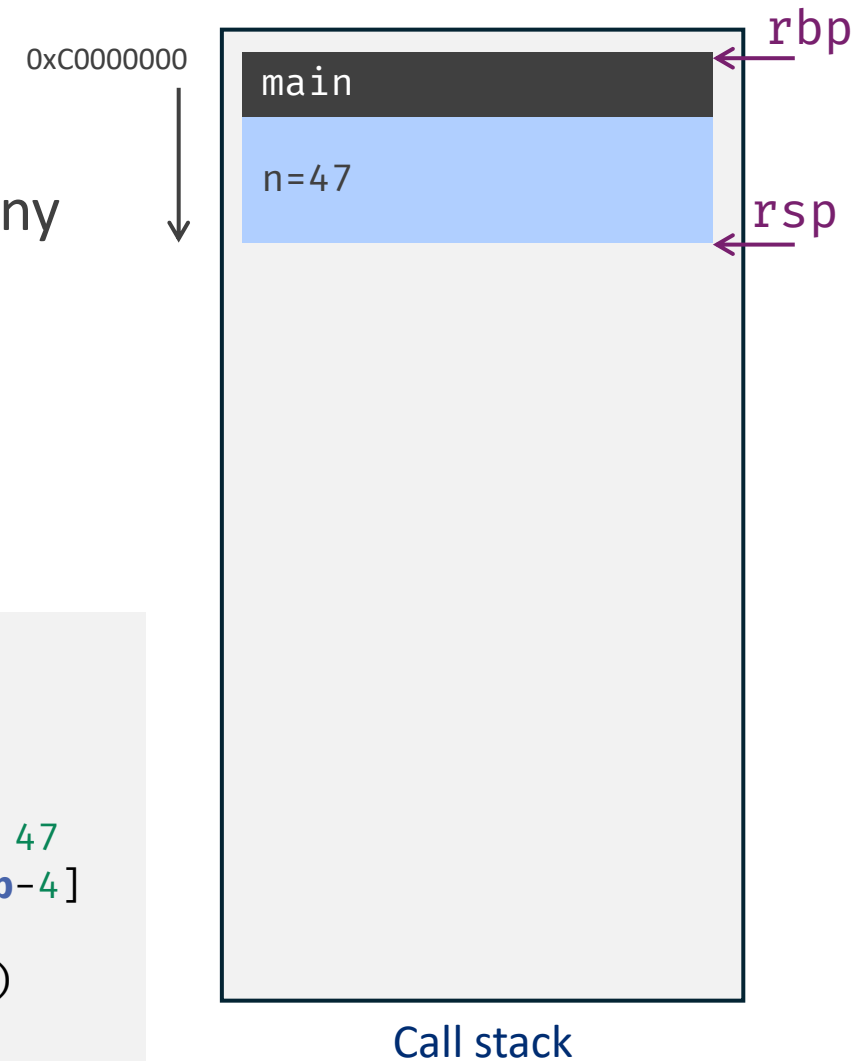


Zásobník – call stack

- Registre rbp a rsp obsahujú adresy ohraničujúce aktuálny rámec
- rbp vidíme pri prístupe k lokálnej premennej
- rsp vidíme pri vytvorení nového rámca

```
16 int main() {  
17     int n = 47;  
18     describeNumber(n);  
19 }
```

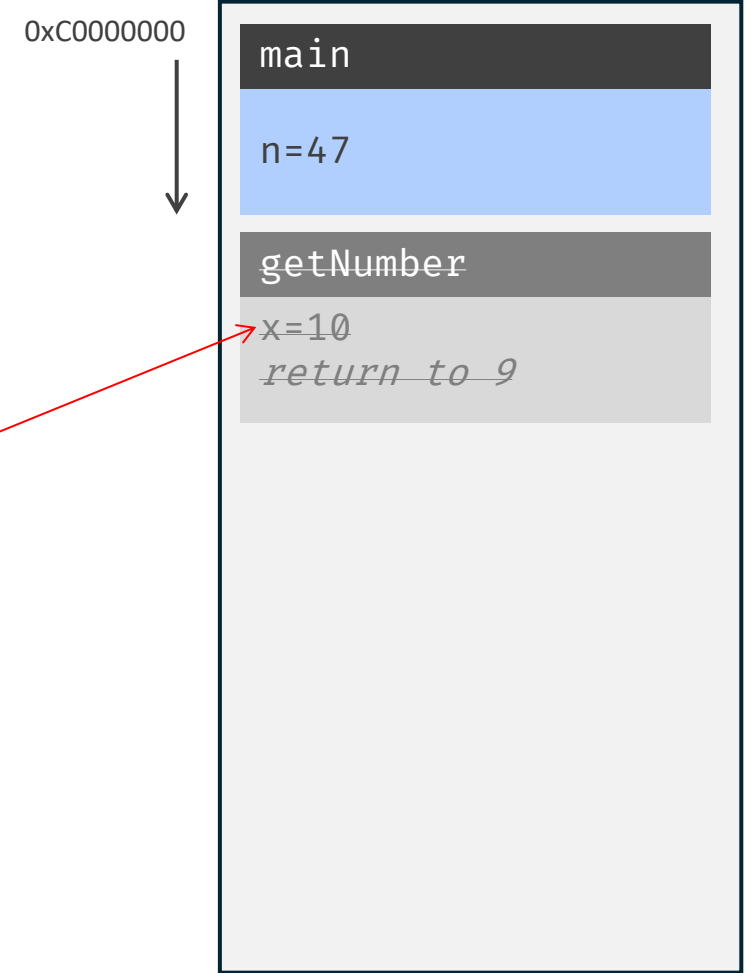
```
main:  
    push    rbp  
    mov     rbp, rsp  
    sub     rsp, 16  
    mov     DWORD PTR [rbp-4], 47  
    mov     eax, DWORD PTR [rbp-4]  
    mov     edi, eax  
    call    describeNumber(int)  
    mov     eax, 0
```



Zásobník

- Z fungovania zásobníka volaní vyplýva, že **najdlhšia možná** životnosť lokálnych premenných je ohraničená časom vykonávania funkcie

```
1 #include <iostream>
2
3 int *getNumber() {
4     int x = 10;
5     return &x;
6 }
7
8 int main() {
9     int *px = getNumber();
10    std::cout << *px << "\n";
11 }
```

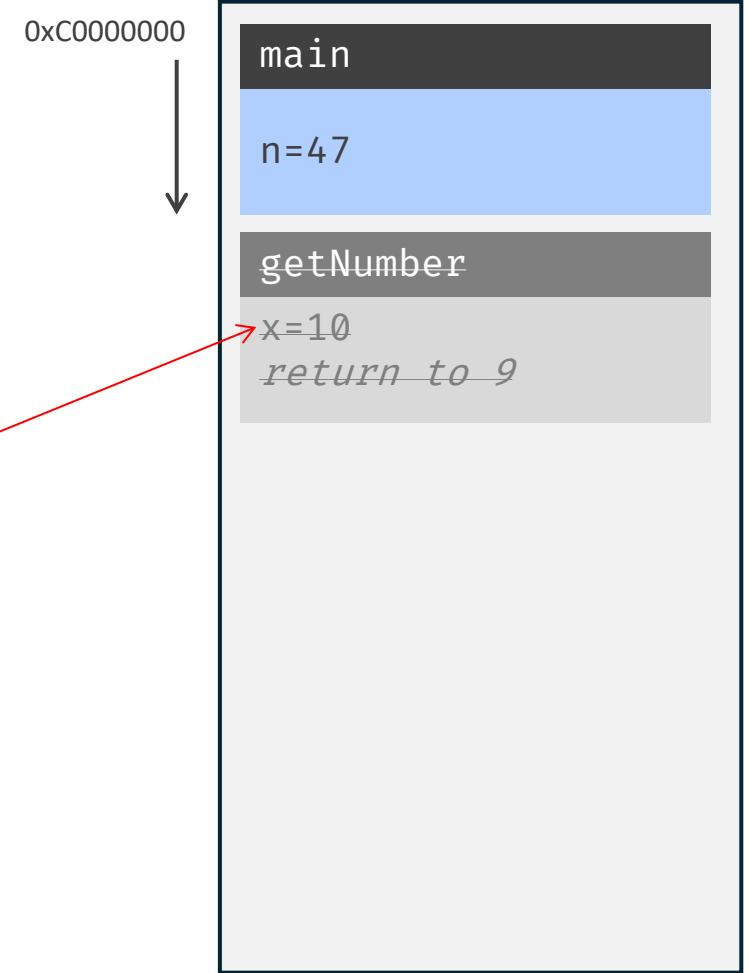


Call stack

Zásobník

- V tomto prípade sme vytvorili **visiaci ukazovateľ** (angl. dangling pointer)
- Dereferencovanie takéhoto ukazovateľa vedie k nedefinovanému správaniu

```
1 #include <iostream>
2
3 int *getNumber() {
4     int x = 10;
5     return &x;
6 }
7
8 int main() {
9     int *px = getNumber();
10    std::cout << *px << "\n";
11 }
```



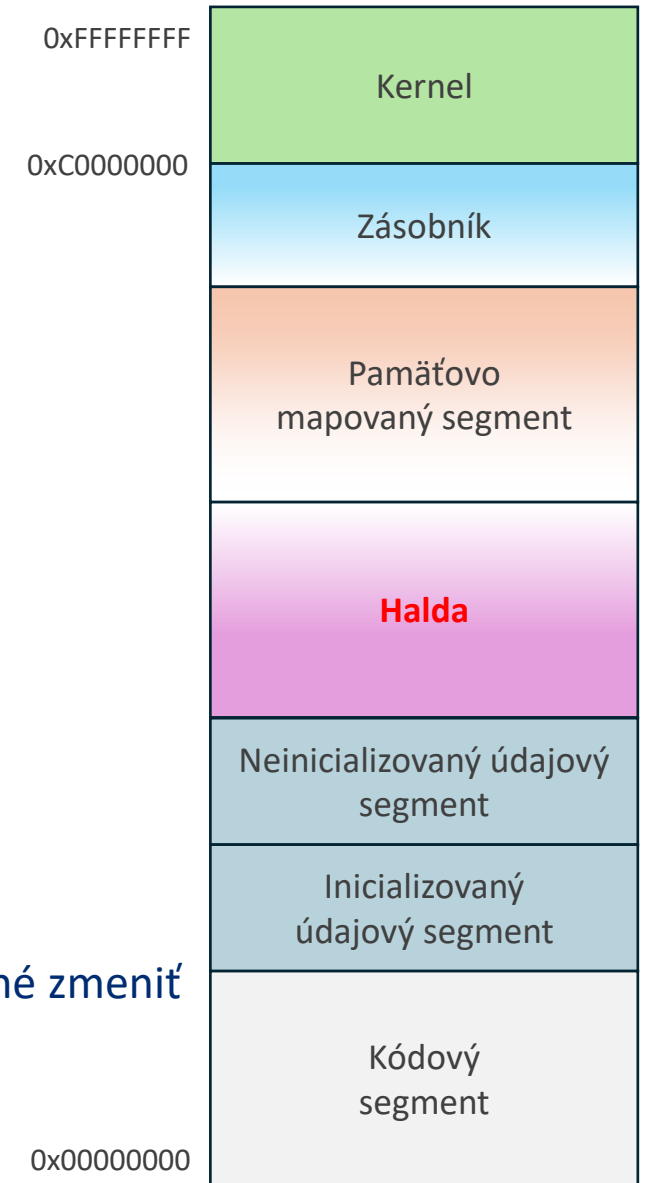
Call stack

Halda

- Obsahuje dynamicky alokované premenné – vytvorené pomocou operátora new – zatiaľ sú to pre nás inštancie tried
- Operátor **new** vráti ukazovateľ na vytvorenú inštanciu
- Veľkosť zásobníka je pomerne malá:
 - 1MB – Windows*
 - 8MB – Linux*
- Na druhej strane, veľkosť haldy je:**
 - *teoreticky* veľmi veľká
 - *a prakticky* veľká – rádovo veľkosť RAM

* Tieto veľkosti je možné zmeniť

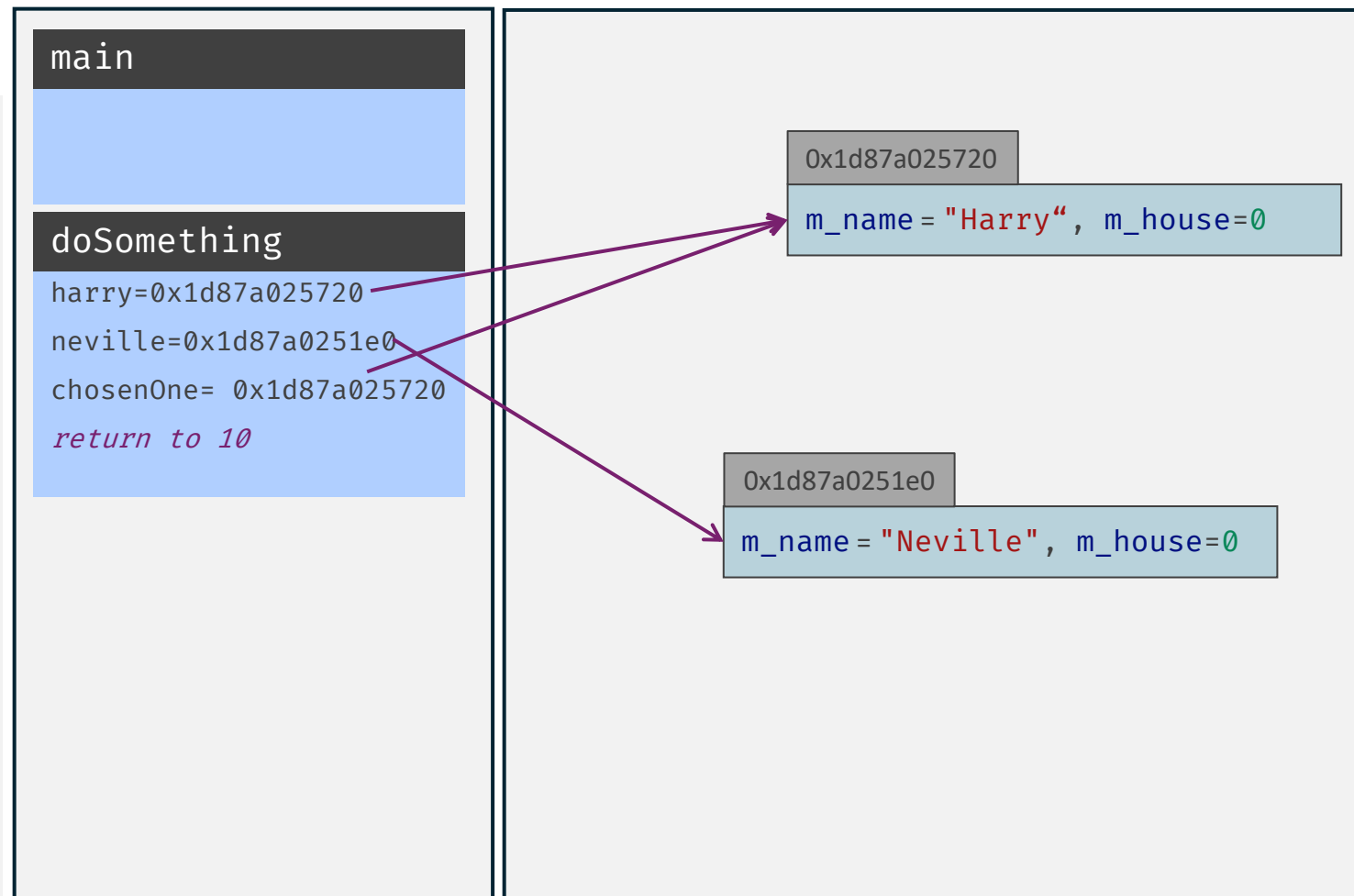
** Viac na 6BI0035



Halda

```
1 void doSomething() {
2     Wizard *harry = new Wizard(
        "Harry",
        House::Gryffindor);
3     Wizard *neville = new Wizard(
        "Neville",
        House::Gryffindor);
4     Wizard *chosenOne = harry;
5     std::cout
        << chosenOne->getName()
        << " is the chosen one.\n";
6 }
7
8
9 int main() {
10     doSomething();
11     std::cout << "The End.\n";
12 }
```

* Tento princíp je vám známy z jazyka Java

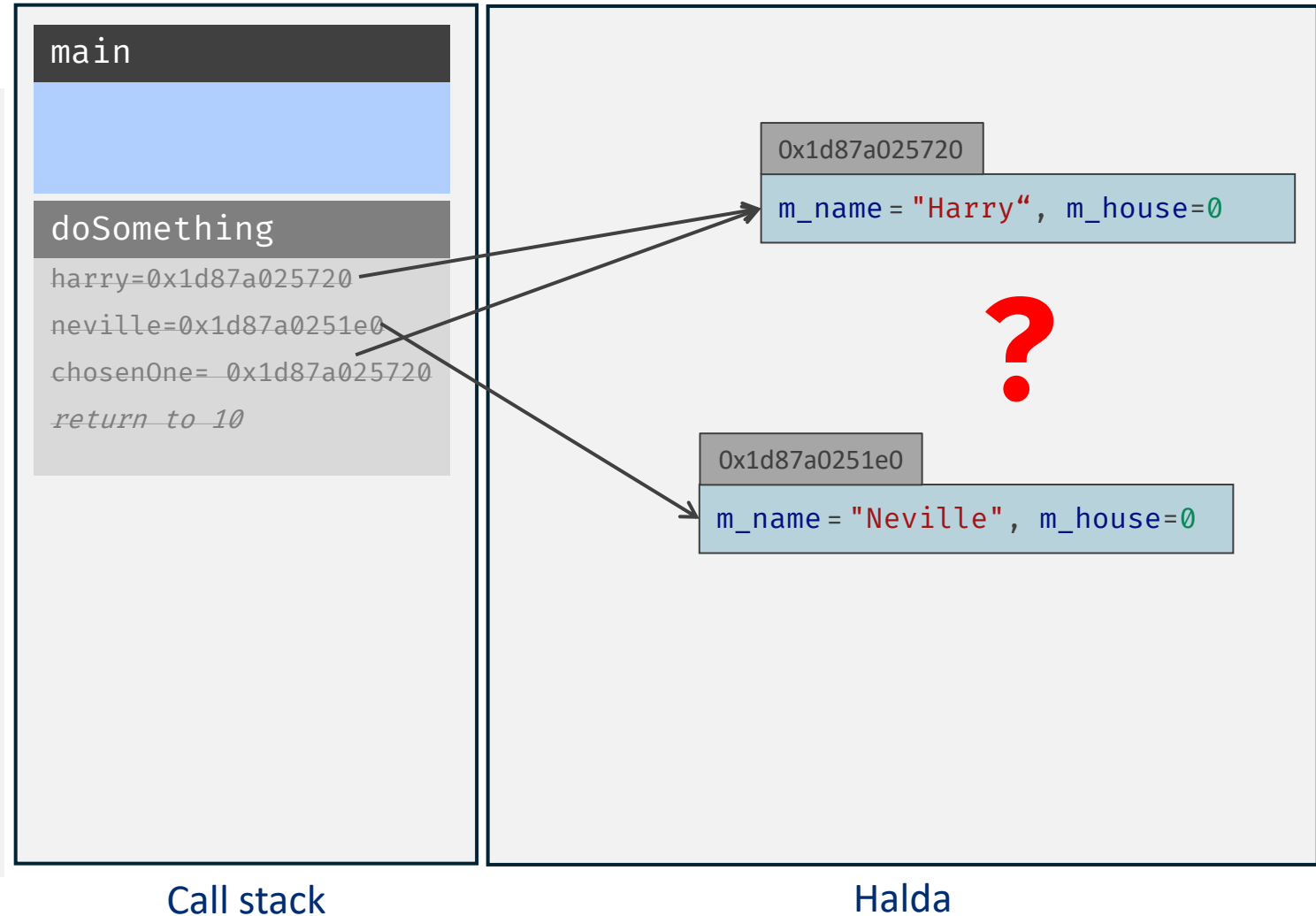


Call stack

Halda

Halda

```
1 void doSomething() {
2     Wizard *harry = new Wizard(
        "Harry",
        House::Gryffindor);
3     Wizard *neville = new Wizard(
        "Neville",
        House::Gryffindor);
4     Wizard *chosenOne = harry;
5     std::cout
        << chosenOne->getName()
        << " is the chosen one.\n";
6 }
7
8
9 int main() {
10     doSomething();
11     std::cout << "The End.\n";
12 }
```

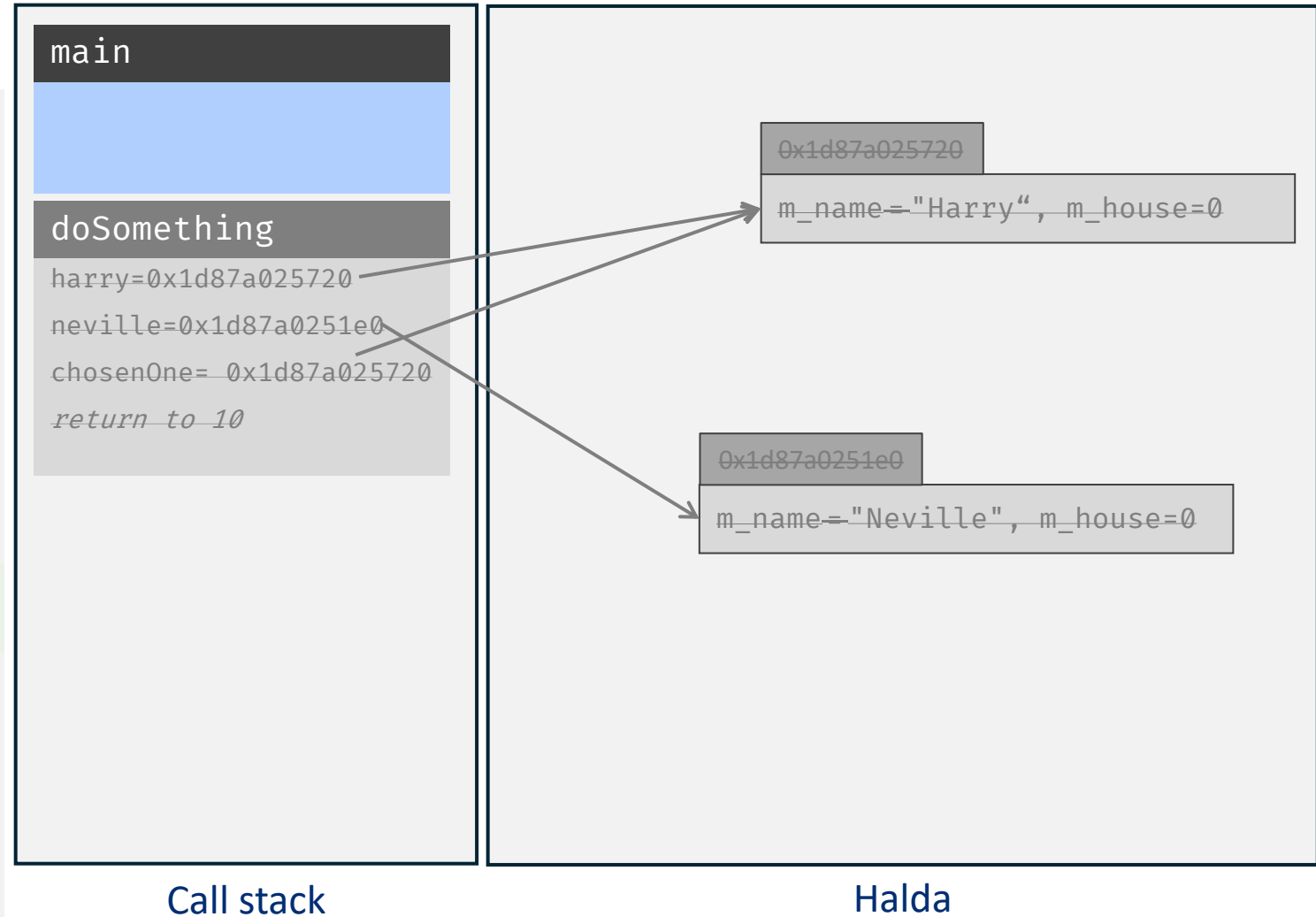


Manuálna správa pamäte

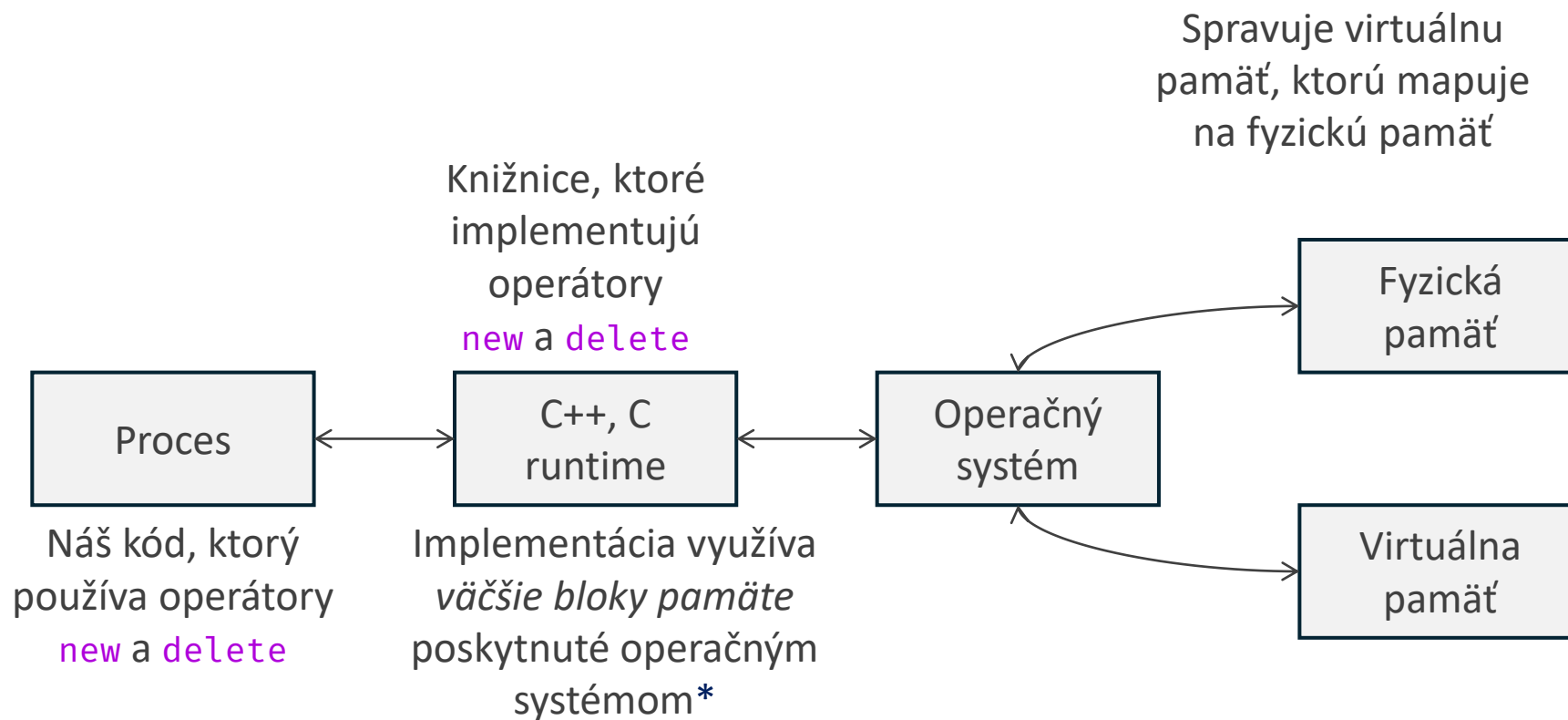
- V jazykoch s **automatickou správou pamäte** (Java, C#) sa o uvoľnenie objektov, ku ktorým už nevedie žiadna referencia (ukazovateľ), postará tzv. Garbage Collector (GC, trochu viac tu)
- GC, podobne ako kompilátor, nie je nič zázračné – je to iba ďalší program, ktorý sa spúšťa spolu s našim programom (napísaným napr. v jazyku Java – súčasť JVM)
- V jazykoch s **manuálnou správou pamäte**, medzi ktoré sa radí aj C++, je uvoľnenie pamäte **zodpovednosťou programátora**
- Opakom operátora **new** je operátor delete, ktorý uvoľní predtým alokovanú pamäť
- **Únik pamäte** (angl. **memory leak**) nastane, ak programátor neuvoľní pamäť a k tejto pamäti už nevedie (ani nepriamo) žiadny ukazovateľ

Halda

```
1 void doSomething() {
2     Wizard *harry = new Wizard(
        "Harry",
        House::Gryffindor);
3     Wizard *neville = new Wizard(
        "Neville",
        House::Gryffindor);
4     Wizard *chosenOne = harry;
5     std::cout
        << chosenOne->getName()
        << " is the chosen one.\n";
6     delete harry;
7     delete neville;
8 }
9
10
11 int main() {
12     doSomething();
13     std::cout << "The End.\n";
14 }
```



Manuálna správa pamäte



* Cieľom je vyhnúť sa častému volaniu funkcií operačného systému

Manuálna správa pamäte

- Po skončení procesu operačný systém uvoľní všetku pamäť, ktorú mu prideliť
- Ponúka sa otázka, či je *naozaj nutné pamäť uvoľňovať priebežne*
- Pri krátko bežiacich programoch to, vskutku, nie je *nevyhnutné*; **avšak**
- Ak proces beží dlho nepoužívaná pamäť sa akumuluje a **odoberá tak zdroje iným procesom**
- Preto, pri alokovaní pamäte je hneď nutné vedieť, kde a kedy sa pamäť dealokuje
- Programátor tak musí mať prehľad o životnom cykle objektov, ktoré za behu jeho programu existujú – **neskôr nám táto znalosť umožní spravovať pamäť automaticky**

Typ odkaz

Angl. reference



Typ odkaz

- Premenná typu odkaz (angl. reference) je **premenná, ktorá sa správa trochu inak**
- Predstavuje **alias** inej premennej/hodnoty – umožňuje pracovať s *odkazovanou hodnotou* (angl. *referenced value*) ako by sme pracovali s pôvodnou
- Spolu s ukazovateľom je to spôsob ako pracovať s premennou **nepriamo** (angl. **indirection**)
- Syntax: <typ> &
- Príklady:
 - `int &` – odkaz na typ `int`
 - `double &` – odkaz na typ `double`
 - `int &&` – odkaz na odkaz neexistuje*

Syntax && je však korektná, ale má iný význam. Viac neskôr. *

Typ odkaz

- Premennú, na ktorú odkazujeme nastavíme práve a len pri definícii odkazu (3), mená `x` a `rx` odkazujú na to isté pamäťové miesto
- Neskoršie priradenie (6) priradzuje do odkazovanej premennej (pamäťového miesta)

```
1 int main() {  
2     int x = 1'478'958;  
3     int &rx = x;  
4     std::cout << rx << "\n"; // 1'478'958  
5     int y = 200;  
6     rx = y;  
7     std::cout << x << "\n"; // 200  
8 }
```

Odkaz a ukazovateľ

- Rovnaké správanie vieme dosiahnuť použitím ukazovateľov
- **Avšak ...**

```
1 int main() {  
2     int x = 1'478'958;  
3     int &rx = x;  
4     std::cout << rx << "\n"; // 1'478'958  
5     int y = 200;  
6     rx = y;  
7     std::cout << x << "\n"; // 200  
8 }
```

```
1 int main() {  
2     int x = 1'478'958;  
3     int *rx = &x;  
4     std::cout << *rx << "\n"; // 1'478'958  
5     int y = 200;  
6     *rx = y;  
7     std::cout << x << "\n"; // 200  
8 }
```

Odkaz a ukazovateľ

```
1 void add(int *pa, int *pb, int *result) {
*2     if (pa && pb && result) {
3         int a = *pa;
4         int b = *pb;
5         *result = a + b;
6     }
7 }
8
9 int main() {
10     int a = 20;
11     int b = 30;
12     int c;
13     add(&a, &b, &c);
14     std::cout << c << "\n"; // 50
15 }
```

```
1 void add(int &pa, int &pb, int &result) {
2     int a = pa;
3     int b = pb;
4     result = a + b;
5 }
6
7 int main() {
8     int a = 20;
9     int b = 30;
10    int c;
11    add(a, b, c);
12    std::cout << c << "\n"; // 50
13 }
```

Využitie implicitnej konverzie. *

Odkaz a ukazovateľ

- Nepoznáme **nulový odkaz**
- Ak funkcia/metóda preberá parameter, ktorý podľa logiky programu nemôže byť nulový **použijeme odkaz** – zodpovednosť za korektnú hodnotu parametra je tak na volajúcom*
- Ak je *nullová* hodnota prípustná (napr. vyjadruje absenciu niečoho, ktorá je súčasťou bežnej logiky programu), môžeme **použiť ukazovateľ** (alebo triedu `std::optional`)

Vyjadrenie logiky v typovom systéme. *

Typ `std::optional`

- Typ `std::optional` umožňuje vyjadriť absenciu hodnoty
- Poskytuje ho štandardná knižnica v hlavičke `<optional>` od štandardu C++17
- Využíva **hodnotovú sémantiku** – jedna z najdôležitejších funkcionálností jazyka C++
 - Podrobne sa jej budeme venovať v nasledujúcich prednáškach
- Najčastejšie sa používa ako typ návratovej hodnoty, avšak je možné použiť ho kdekoľvek
- Umožňuje zmeniť čiastočnú funkciu (napr. celočíselné delenie) na úplnú funkciu
 - Z pohľadu teórie typov ide o sum type


```

#include <iostream>
#include <optional>

std::optional<int> div(int a, int b) {
    if (b == 0) {
        return std::nullopt;
    }
    return std::optional<int>(a / b);
}

int main() {
    int a = 20;
    int b = 0;
    std::optional<int> result = div(a, b);
    if (result.has_value()) {
        std::cout << "The result is " << *result << "\n";
    } else {
        std::cout << "The division has no result!\n";
    }
}

```

Typ pole

Angl. array

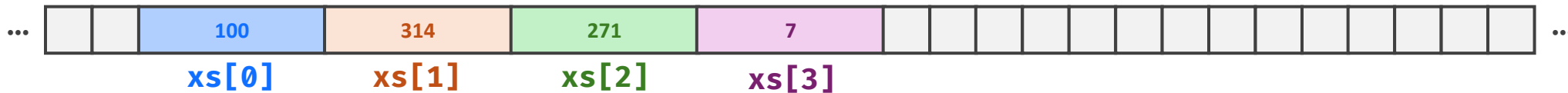
Typ pole

- Typ pole je súčasť jazyka podobne ako fundamentálne typy
- Syntax: `<typ> <identifikátor>[<veľkosť>]`
- Veľkosť poľa musí byť známa pri preklade
- K prvkom poľa pristupujeme pomocou operátora `[]`
- Príklady:
 - `int xs[10];` – pole celých čísel o veľkosti 10
 - `Wizard *ws[4];` – pole ukazovateľov na inštanciu triedy `Wizard` o veľkosti 4

Pamäťová reprezentácia

```
int main() {  
    int xs[4] = {100, 314, 271, 7};  
    for (int i = 0; i < 4; ++i) {  
        std::cout << xs[i] << "\n";  
    }  
    std::cout << sizeof(xs) << "\n"; // ?  
}
```

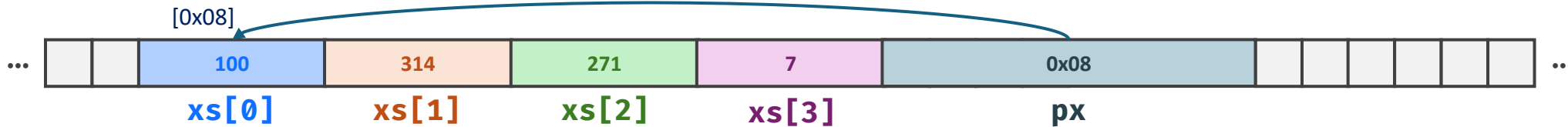
- Operátor `sizeof` vráti veľkosť argumentu (typ alebo hodnota) v bajtoch
- V tomto prípade je argumentom pole hodnôt typu `int` o veľkosti 4 bajty
- Počet prvkov poľa `sizeof(xs) / sizeof(int)`



Implicitné konverzie

- Pole je implicitne konvertovateľné na ukazovateľ na typ prvkov poľa

```
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    std::cout << *px << "\n"; // ?  
}
```



Práca s poľom

- Syntax práce s poľom môže byť matúca

```
void zero_array(int xs[4]) {  
    for (int i = 0; i < 4; ++i) {  
        xs[i] = 0;  
    }  
}  
  
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    zero_array(xs);  
    std::cout << *px << "\n"; // ?  
}
```

Práca s poľom

- Syntax práce s poľom môže byť mätúca

```
void zero_array(int xs[]) {  
    std::cout << sizeof(xs) << "\n"; // ?  
    for (int i = 0; i < 4; ++i) {  
        xs[i] = 0;  
    }  
}  
  
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    zero_array(xs);  
    std::cout << *px << "\n";  
}
```

Práca s poľom

- Syntax práce s poľom môže byť mätúca

```
void zero_array(int *xs) {  
    std::cout << sizeof(xs) << "\n"; // 8  
    for (int i = 0; i < 4; ++i) {  
        xs[i] = 0;  
    }  
}  
  
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    zero_array(xs);  
    std::cout << *px << "\n";  
}
```


Práca s poľom

- Ak pole uvedieme ako typ parametra, vždy sa implicitne konvertuje na ukazovateľ
- Preto nie je možné získať jeho veľkosť operátorom `sizeof`
- Veľkosť je nutné poslať ako samostatný parameter
- Operátorom `sizeof` môžeme veľkosť získať iba v bloku, kde je pole definované

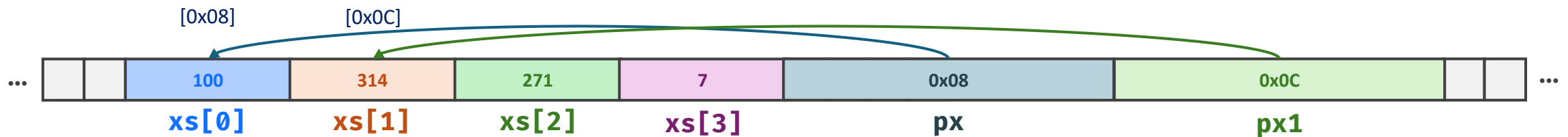
```
void zero_array(int *xs, int length) {  
    for (int i = 0; i < length; ++i) {  
        xs[i] = 0;  
    }  
}  
  
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    zero_array(xs, sizeof(xs) / sizeof(int));  
    std::cout << *px << "\n";  
}
```

- Takéto použitie je bežné v jazyku C
- V jazyku C++ používame triedy `std::array` a `std::vector`
- Vstavané polia používame *rozumne* – ako lokálne premenné alebo atribúty

Aritmetika ukazovateľov

- K ukazovateľom môžeme pripočítavať celé čísla
- Pri výpočte sa berie do úvahy typ, na ktorý ukazovateľ ukazuje
- Ak, napríklad pripočítame 1 k ukazovateľu na typ `int`, výsledok je adresa vyššia o `sizeof(int)` bajtov

```
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    int *px1 = px + 1;  
}
```



Aritmetika ukazovateľov

- Aritmetiku spojenú s ukazovateľom na začiatok poľa a dereferencovaním môžeme použiť na prístup k prvku poľa (5) ako alternatívu ku klasickému prístupu (4)
- `operátor[]` použitý s ukazovateľom (6) je iba skráteným zápisom aritmetiky spojenej s dereferencovaním
- Tento mechanizmus nám zároveň ukazuje, prečo väčšina programovacích jazykov indexuje od 0 – je to prirodzená vlastnosť **nízkoúrovňových polí**

```
1 int main() {  
2     int xs[4] = {100, 314, 271, 87};  
3     int *px = xs;  
4     std::cout << xs[2] << "\n";      // 271  
5     std::cout << *(px + 2) << "\n";  // 271  
6     std::cout << px[2] << "\n";      // 271  
7 }
```

Aritmetika ukazovateľov

- Identifikátor poľa (*xs*) môžeme priamo použiť v aritmetickom výraze – prebehne implicitná konverzia na ukazovateľ
- V príklade (6) používame túto syntax na prístup k prvku na indexe 2

```
1 int main() {  
2     int xs[4] = {100, 314, 271, 87};  
3     int *px = xs;  
4     std::cout << xs[2] << "\n";      // 271  
5     std::cout << px[2] << "\n";      // 271  
6     std::cout << *(xs + 2) << "\n";  // 271  
7     std::cout << *(px + 2) << "\n";  // 271  
8     std::cout << (*px + 2) << "\n";  // 102  
9     std::cout << (px + 2) << "\n";  // 0x7ffd8c2cad78  
10 }
```

Nesprávny prístup do pamäte

- Aritmetika ukazovateľov nám umožňuje veľmi ľahko pokúsiť sa pristúpiť k pamäti, ktorá nám nepatrí
- Toto vedie k už spomínanému nedefinovanému správaniu a spôsobilo nespočetné množstvo incidentov
- Program v tom bode môže ale nemusí skončiť chybou

```
int main() {  
    int xs[4] = {100, 314, 271, 87};  
    int *px = xs;  
    std::cout << *(px + 4) << "\n"; // ?  
    std::cout << *(px + 1'000'000) << "\n"; // ?  
}
```

Technická realizácia poľa na zásobníku

```
#include <iostream>

int main() {
    int xs[4] = {100, 314, 271, 87};
    for (int i = 0; i < 4; ++i) {
        std::cout << xs[i];
    }
}
```

<https://godbolt.org/z/5Esh735sb>

*! Príklad na rámec prednášky

```
"main":
    movabs    rax, 1348619731044
    push     rbx
    sub      rsp, 16
    mov      QWORD PTR [rsp], rax
    mov      rbx, rsp
    movabs    rax, 373662155023
    mov      QWORD PTR [rsp+8], rax

.L2:
    mov      esi, DWORD PTR [rbx]
    mov      edi, OFFSET FLAT:"std::cout"
    add      rbx, 4
    call     "std::basic_ostream::operator<<(int)"
    lea      rax, [rsp+16]
    cmp      rbx, rax
    jne      .L2
    add      rsp, 16
    xor      eax, eax
    pop      rbx
    ret
```

Vytvorenie (polovice) poľa v 64 bitovom registri `rax`:

```
movabs    rax, 1348619731044
```

$(1348619731044)_{10}$

$(00000000\ 00000000\ 00000001\ 00111010\ 00000000\ 00000000\ 00000000\ 01100100)_2$

$(01100100\ 00000000\ 00000000\ 00000000\ 00111010\ 00000001\ 00000000\ 00000000)_2$ little-endian

$(01100100\ 00000000\ 00000000\ 00000000\ 00111010\ 00000001\ 00000000\ 00000000)_2$ little-endian

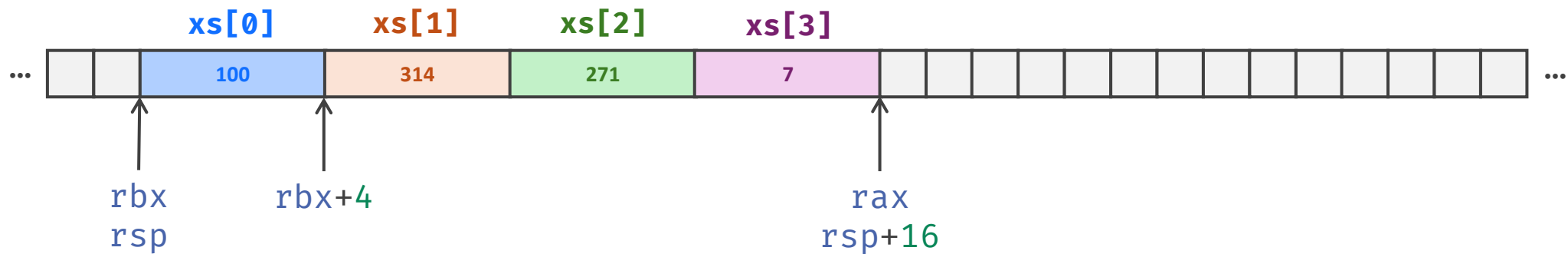
int 100 little-endian

int 314 little-endian

Zápis (polovice) poľa do zásobníka:

```
mov       QWORD PTR [rsp], rax
```

Rovnaký postup sa zopakuje pre druhú polovicu poľa.



```

mov     esi, DWORD PTR [rbx]
mov     edi, OFFSET FLAT:"std::cout"
add     rbx, 4
call    "std::basic_ostream::operator<<(int)"
lea     rax, [rsp+16]
cmp     rbx, rax
jne     .L2

```

`add rbx, 4` zvýši hodnotu `rbx` o 4 – posunie adresu o 4 bajty – teda na ďalší `int` (aritmetika ukazovateľov ako bola ukázaná tu neexistuje – assembler nepozná typy, všetko sú čísla)

`cmp rbx, rax` porovná `rbx` s koncom rámca
`jne .L2` Ak sa nerovná, cyklus sa opakuje – skok na začiatok kódu cyklu

`rbx` obsahuje adresu (prakticky ukazovateľ), čísla, ktoré sa ide aktuálne vypisovať, inicializuje sa na adresu začiatku rámca, ktorý zodpovedá začiatku poľa (`mov rbx, rsp`)
`rax` obsahuje adresu konca rámca

`operator<<` je na pozadí obyčajná metóda triedy, ktorá potrebuje 2 parametre, ktoré sa odovzdajú v registroch `edi` (1. parameter) a `esi` (2. parameter).
V `edi` sa odovzdáva `this` (ďalšia prednáška) a v `esi` číslo, na ktoré ukazuje `rbx`.

Zhrnutie

- **Ukazovateľ** – adresa inej premennej – spôsob nepriameho prístupu
 - **Aritmetika ukazovateľov** – umožňuje prístup k prvkom poľa
- **Odkaz** – jednoduchšia a bezpečnejšia alternatíva ukazovateľov
- Pamäťový priestor procesu:
 - **Zásobník** – rámce
 - **Halda** – manuálna správa pamäte
- **Polia**
 - Jednoduchá súčasť jazyka
 - V C++ zvyčajne používame alternatívy (kontajnery zo štandardnej knižnice)

Ďakujem za pozornosť

Priestor na vaše otázky

